Assignment 3

Modular Architecture and Package Implementation

Total Points: 5 **Submission Format:** .zip archive containing the project directory. **Prerequisites:** Python Modules, File I/O, Exception Handling.

1. Objective

The objective of this assignment is to demonstrate competency in **modular programming**. Students must refactor monolithic code into a structured **Python Package**, demonstrating a clear Separation of Concerns (SoC) between business logic (mathematical computations) and presentation logic (user interface).

2. Technical Specifications

You are required to implement a directory structure that constitutes a valid Python package. The application will function as a geometric calculator.

2.1 Directory Hierarchy

Your submission must adhere strictly to the following file system structure:

```
StudentName_ID_Assignment12/        <-- Root Directory
│
├── main_application.py             <-- Entry Point (User Interface)
│
└── geometry_pkg/                   <-- The Python Package
    ├── __init__.py                 <-- Package Initializer
    └── areas.py                    <-- Computation Module
```

2.2 Module Requirements: geometry_pkg/areas.py

This module acts as the backend library. It must be a **pure function** module.

- **Constraint:** This file must **not** contain any input() or print() statements. It should only accept arguments and return values.
- **Functions to Implement:**
  - calculate_circle_area(radius): Returns the area (πr^2)
  - calculate_rectangle_area(length, width): Returns the area ($l \times w$)
  - calculate_triangle_area(base, height): Returns the area ($0.5 \times b \times h$)
- **Error Handling:** If any negative parameters are passed, the functions must raise a ValueError.

2.3 Package Initialization: geometry_pkg/__init__.py

This file establishes the directory as a package. While it may remain empty, you are encouraged to use it to expose the functions from areas.py to the package namespace if you wish to demonstrate advanced understanding.

2.4 Application Entry Point: main_application.py

This script acts as the frontend client.

- **Import Strategy:** You must import the functionality from your custom package (e.g., from geometry_pkg import areas).

- **Runtime Logic:**
  - Display a CLI menu allowing the user to select a shape or exit.
  - Accept user input and cast it to the appropriate data type (float).
  - Invoke the corresponding function from the geometry_pkg package.
  - **Exception Handling:** Implement try...except blocks to handle:
    - ValueError (for non-numeric inputs or negative dimensions).
    - General exceptions.

3. Grading Rubric (5 Points)

CRITERIA	POINTS	DESCRIPTION
PACKAGE STRUCTURE	**1.0**	Directory hierarchy allows for successful import of geometry_pkg. __init__.py is present.
ENCAPSULATION	**1.0**	areas.py contains **only** logic (no I/O). The separation between interface and logic is respected.
CORRECTNESS	**1.0**	Mathematical formulae are implemented correctly using Python operators.
ERROR HANDLING	**1.0**	The application catches invalid inputs (e.g., strings entered as numbers, negative dimensions) without crashing.
EXECUTION	**1.0**	The main_application.py script runs the interactive loop and terminates cleanly upon user request.

4. Sample Execution Trace

*** Geometric Area Calculator ***
1. Circle
2. Rectangle
3. Triangle
4. Quit

Select operation: 1
Enter radius: -5
Error: Dimensions cannot be negative.

Select operation: 2
Enter length: 10
Enter width: 5.5
Result: The area is 55.00